

Notes (Week 3 Monday)

These are the notes I wrote during the lecture.

Choose your own adventure!

1. wrap up last week
(functions, logic)
2. wrap up functions, ignore logic
3. move on to w3 topic
(induction, recursion)

Quickly, functions.

Functions are a special case of
binary relations.

Binary relations
are a special case of sets.

Specifically, a binary relation R
is a set of the form
 $R \subseteq S \times T$
or in other words, it's a set
of pairs.

E.g., $\leq$ is the set of all pairs
of numbers (x,y) such that y
is no less than x .

Functions f from S to T can be seen
as relations
 $f \subseteq S \times T$

$(x,y) \in f$ means that $f(x) = y$

In order for a relation to count
as a function, for every $x \in S$,
there exists *exactly one* $y \in T$
such that $(x,y) \in f$

In words: every input must produce
exactly one output.

Injective functions

A function is *injective* if

whenever $x_1 \neq x_2$,

$$f(x_1) \neq f(x_2)$$

An injective function maps distinct elements to distinct elements.

Let's say these are both $\mathbb{N} \rightarrow \mathbb{N}$

$$f(x) = x + 1 \quad \text{injective}$$

^ the *range* of this function is the positive $\mathbb{N}$ s

$$f(x) = 0$$

^ the *range* of this function is $\{0\}$

Surjective functions

$$f \subseteq S \times T$$

S the set of inputs (domain)

T the set of outputs (codomain)

range the outputs that actually happen.

A function is *surjective* if the codomain equals the range.

Or: if every output is possible.

$$f(x) = \begin{cases} x - 1 & \text{if } x > 0 \\ 124894 & \text{if } x = 0 \end{cases}$$

$$f(124895) = 124894$$

$$f(0) = 124894$$

^ this happens to be surjective, but not injective.

Bijjective = injective + surjective

$$f(x) = x$$

If a function is bijective, it has an *inverse*.

Intuitively, a way to "undo" the function.

If $f : S \rightarrow T$ is bijective,

$f^{-1} : T \rightarrow S$ (its inverse) exists, and has the property

$$f^{-1}(f(x)) = x$$

To construct the inverse function,
you take the converse of the relation
that characterises the function.

If $R \subseteq S \times T$ is a relation,
the **converse** of R ,
written $R^{\leftarrow} \subseteq T \times S$ defined as

$$\{(t,s) \mid (s,t) \in R\}$$

In general, the converse of a function
is not a function.

But for bijective functions,
the converse **is** a function.

Q: What happens if I leave out the
base case in the program?

A: Stack overflow
eventually, we run out of memory
to represent the stack frames

A: Go on forever

A: Integer overflow

Q: What goes wrong in maths?

A: You might be describing a property
that many functions, instead of
a specific one.

A: for induction purposes
(in order for inductive proofs
to be valid)

$$f(x) = 1 + f(x)$$

If I allow the above definition,
cancel $f(x)$ on both sides,
 $0 = 1$

In programming, the world doesn't
end if I forget a base case.
In mathematics, allowing such
definitions can end the world.

$$(1) \ 0 \in S$$

$$(2) \text{ If } n \in S \text{ then } n+1 \in S$$

There are many sets that satisfy
the properties (1) and (2) above.

$\mathbb{N}$ satisfies them.

$\mathbb{Z}$ also does

$\mathbb{R}$ also does

....

Q: What is the definition
of a definition?

A: A definition is a property
such that only one object
has that property.

Example:

Let π be the number s.t.

$c = \pi * d$ for every circle

Non-example:

Let x be the integer that is
greater than 5.

When we say

S is inductively defined by:

(1) $0 \in S$

(2) If $n \in S$ then $n+1 \in S$

What we really mean is:

S is the **smallest** set that
satisfies (1) and (2).

By smallest I mean wrt subset
inclusion.

S is the (unique) set that
satisfies (1) and (2), and,
for every **other** set T that
satisfies (1) and (2),
 $S \subseteq T$.

How do we know that this S exists?

A: take the intersection of every
set that satisfies (1) and (2)

Q: Aren't we just defining sets
in terms of stuff we already had?
Are we going on circles?

We do generally presuppose the
existence of **something**, but not
necessarily numbers.

Suppose all I have is sets.

How do I make numbers?

$0 = \{\}$

$1 = \{\{\}\}$

$2 = \{\{\}, \{\{\}\}\}$

A number n is the union of all numbers less than n .

Q: Is it **always** the case that the intersection construct above satisfies the clauses and/or is unique

The clauses in an inductive definition we require:

- the conclusion of every clause has the form $x \in T$, where T is the set we're defining
- if you mention T in the premises, it must be a **positive** mention

$x \in T \Rightarrow x+1 \in T$ $\leftarrow$ positive premise on T

$x \notin T \Rightarrow x+1 \in T$ $\leftarrow$ negative premise on T

As long as I obey the above two constraints, then the least set satisfying the clauses always exists.

As a trivial example:

$x \notin T \Rightarrow x \in T$ what does this "definition" mean?

A: no set satisfies this clause.

Q: So why does the intersection of the sets that satisfy positive clauses always satisfy the clauses?

A: Positive clauses have a property called **monotonicity**

A function on sets is **monotonic** if

whenever $S \subseteq T$ then $f(S) \subseteq f(T)$

Imagine f is the "apply the clauses once" function, and S is the intersection.

"What if you forget the base case"

For mathematical functions: you introduce contradictions

In programming: you may introduce bugs

For inductive sets: you just end up defining the empty set.

$n \in \mathbb{N} \Rightarrow n+1 \in \mathbb{N}$ -- the empty set satisfies this

In general, inductive (or recursive) definitions of sets (or functions) can have

- many base cases
- many inductive cases

The C struct

```
struct node {
    int data;
    struct node* next;
}
```

looks like a recursive definition, but it isn't quite one.

Elements of a recursive definition must be iteratively constructed from smaller elements, but pointers can point anywhere.

```
// that's an infinite linked list
// whose every element is 2141
node foo;
foo.data = 2141;
foo.next = &foo;
```

Recursively defined objects (in maths) cannot have this kind of circularity.

$$\Sigma = \{h, i\}$$

Prove that $hi \in \Sigma^*$

1. By (B) $\lambda \in \Sigma^*$
2. By (R) and (1), $i\lambda = i \in \Sigma^*$
3. By (R) and (2), $h \cdot i = hi \in \Sigma^*$

In this course,

we often write $\top$ "top" to mean "true"
and $\perp$ "bottom" to mean "false"

Because we think of the wffs as an inductive definition, some things are ruled out by construction.

We can't write an infinite conjunction

$$\phi_0 \wedge \phi_1 \wedge \dots$$

Formulas can't have infinite nesting depth

```
hi·hello
= (R)
h·(i·hello)
  (R)
=  $\lambda \cdot (h \cdot (i \cdot \text{hello}))$ 
```

```

    (by def of single-character concatenation)
=  $\lambda \cdot (h \cdot \text{ihello})$ 
    (by def of single-character concatenation)
=  $\lambda \cdot \text{hihello}$ 
    (B)
= hihello

```

Remember that formulas don't have truth values,
but in particular states (aka valuations) they
have truth values.

A valuation gives values to all variables.

"PROP" is the set of variable names for boolean values.

A valuation is a function from PROP to truth values
 $\{\text{True}, \text{False}\} = \mathbb{B}$

$\text{eval} : (\text{wff}, \text{PROP} \rightarrow \mathbb{B}) \rightarrow \mathbb{B}$

```

eval( $\top$ , v) = True
eval( $\perp$ , v) = False
eval(p, v) = v(p)
eval( $\neg\phi$ , v) =
    if eval( $\phi$ , v) = True then
        False
    else
        True
...

```